

Smart Commerce AI Architecture - Complete Documentation

Welcome to the Smart Commerce AI project! This document serves as the complete technical bible for the repository. Whether you are a new developer onboarding, an investor, or a team member trying to run the project, this guide explains exactly what this system is, how to run it, and the deep technical architecture behind every microservice.

1. Quick Start Guide (How to Clone & Run)

Our goal is to make setting up this massive AI infrastructure as easy as a single command. You do **not** need to install Python, Node.js, or configure environment variables locally. Everything is containerized.

Prerequisites

1. Install Git
2. Install Docker Desktop

Step-by-Step Setup

1. **Clone the Repository:**

```
git clone https://github.com/deepakmuvva99/ai-smart-commerce.git
cd ai-smart-commerce
```

2. **Launch the Architecture:**

```
docker-compose up -d --build
```

Note: The first build will take several minutes as it downloads PyTorch, Node.js, and builds the isolated Linux environments.

3. **Access the Platform:** Open your browser to `http://localhost`

4. **Shutdown the Platform:**

```
docker-compose down
```

2. Authentication & User Workflows

We utilized strict backend validation logic (via Pydantic) to ensure safe user data.

How to Sign Up (Registration)

To create a new user account, you must satisfy the API constraints: - **Email Constraint:** Must be a structurally valid email (e.g., `user@domain.com`). - **Password Constraint:** Must be precisely **8 characters or larger** to prevent Bcrypt processing attacks.

Example Signup Data for Testing: - **Email:** `testuser@example.com` - **Password:** `password123` - **Confirm Password:** `password123`

Once registered, the backend intercepts the signup request and automatically sets `is_verified = True` in the database so you can immediately begin testing the platform.

How to Log in (Admin vs User)

The system employs standard Bearer JSON Web Tokens (JWT). When you sign in, your token determines your role-based access control.

- **Standard Demo User:** Use your newly generated `testuser@example.com` to explore the frontend UI, view products, and simulate traffic metrics.
 - **Admin Dashboard Access:** To explore the Machine Learning metrics, you must log in with the hardcoded Admin Root Account.
 - **Email:** `admin@smartcommerce.com`
 - **Password:** `admin123`
-

3. Database Architecture (Schemas & Constraints)

The Core Backend relies on a robust relational SQLite database (`prod_app.db`) mounted directly to a Docker volume (`db_data`) mapped via SQLAlchemy ORMs.

Primary Entities & Constraints

1. **users Table:**
 - `email` (String, UNIQUE, Indexed): Prevents duplicate accounts.
 - `role` (String, Default: `user`): Dictates Admin vs Standard permissions.
 - `is_verified` (Boolean, Default: `True`): Allows instant logins.
 - *Relationship:* Cascades (Deletes) mapped `carts` and `orders` if a user is wiped.
2. **products & product_variants Tables:**
 - `base_price` & `cost_price`: Floating constraints preventing negative values. Used by the ML models to ensure we never price a product below our manufacturing expense.
 - `inventory`: This is a `@property` aggregation tracking total S/M/L stock volumes across all tied `variants` linked by Foreign Keys.
3. **traffic_logs Table:**

- Used directly by our ML system. Every time a user clicks or previews a product, it logs `event_type` and `product_id`. The Deep Learning engine groups this specific timestamp data to calculate future surges.
4. **ai_metrics & price_history Tables:**
- `predicted_demand` & `reward`: Traces exact decisions made by the RL Agent so administrators can visualize the performance algorithm directly on the dashboard over time.
-

4. Deep Machine Learning Architecture

At its core, **Smart Commerce** is an autonomous e-commerce engine that removes human analysts entirely, relying on continuous multi-stage intelligence streams.

Phase 1: Deep Learning (DL) - The BiLSTM Predictor

File location: `ai_service/Transformer_model/` - **What it does:** It predicts the exact future demand (sales volume) for any given product over the next 24 hours. - **How it works:** We trained a Bidirectional Long Short-Term Memory (BiLSTM) network. Unlike regular neural networks that only look at data backward, BiLSTMs analyze sequential transaction history in both directions simultaneously. By feeding it historical `traffic_logs` and `order_items`, the network learns complex consumer trends (like spikes on weekends or during sales) and produces a raw `predicted_demand` output (e.g., 6.4 predicted sales).

Phase 2: Reinforcement Learning (RL) - The SAC Agent

File location: `ai_service/sac_rl/` - **What it does:** It chooses how to manipulate the product price uniquely based on the BiLSTM's demand prediction. - **How it works:** We use a Soft-Actor Critic (SAC) algorithm. This is an "Actor" model. It reads the State array (Current Price, Base Price, Inventory level, Traffic size, and the *DL Predicted Demand*). It then autonomously explores by shifting a continuous action vector (`price_multiplier`). - *Example Case:* If the DL predicts huge demand (150 sales), but Inventory is critically low (10 items), the SAC RL Agent recognizes this scenario based on past rewards and forces the `price_multiplier` to 1.15x to drastically raise prices, restrict volume, and maximize overall profit. - **The Reward Cycle:** Every few minutes, the `core_backend` scheduler runs, checks how much real revenue the system generated from the recent price change, and fires an HTTP `/reward` payload down to the `ai_engine`. The SAC agent consumes this experience to iteratively rewrite its policy network toward higher accuracy.

5. Security Summary

By decoupling the architecture into containerized microservices, we achieved an enterprise-grade structure. - **Internal Networks:** The AI Engine (Port 8001) and Backend (Port 8000) **do not exist** on the host. An external attacker scanning your IP address cannot see or contact them. - **Proxy Gatekeeper:** The only path inward is through Nginx via Port 80, traversing the `/api/*` directory, which strictly filters and masks CORS origin paths.